Pretrained 2D Object Detectors from COCO to KITTI: Accuracy, Speed, and Domain Shift

CMPE 249 Homework — Track A (Experimental). João Lucas Veras. Code, configurations, and command log: github.com/JoaoLucasVeras/2d-object-detection-evaluation (branch `track-a-evaluation`).

1. Goal and summary

Question. How do modern pretrained detectors from three families compare in accuracy, latency, and memory, and how well do COCO-trained models transfer to autonomous-driving images without retraining?

I evaluated **five COCO-pretrained configurations from three detector families**: YOLO11n and YOLO11s (one-stage, anchor-free), RT-DETR-L (query-based transformer, NMS-free), and Faster R-CNN with a MobileNetV3-320 or ResNet-50-FPN-v2 backbone (two-stage, anchor-based). Each one ran on **COCO val2017 (5,000 images)** and on a **KITTI 2D-object validation split (1,496 images)** on an RTX 3060 Ti. I ran every number in this report myself. Predictions were scored with the official COCO evaluator (pycocotools). Latency was measured per image with CUDA synchronization.

Main findings.

1. My COCO numbers reproduce the published ones to within 0.3–1.5 AP (Table 2). The detection budget explains only 0.1 AP of the YOLO gap (Section 5.4). Source inspection points to multi-label NMS in Ultralytics' validator as the likely remainder.
2. Moving to KITTI costs 12.6–15.3 AP on the three shared classes. The loss is very uneven: **car AP is unchanged or higher**, **person AP roughly halves**, and **truck AP collapses** because many KITTI trucks are predicted as COCO *bus* or *train*. That is largely a taxonomy effect rather than a missed object.
3. On KITTI's 1242×375 frames, the nominal "640" input leaves YOLO only 640×193 pixels of image. Running YOLO at 1280 px raises KITTI AP by 4.6–6.3 points for about 2 ms, and **YOLO11n at 1280 beats YOLO11s at 640**. RT-DETR's stretched input is *not* the reason for its small-object lead: preserving the aspect ratio improves its small-object AP.
4. Latency at batch 1 is overhead-bound. YOLO11n and YOLO11s differ by less than 1 ms, while two-stage Faster R-CNN is the slowest for its accuracy. RT-DETR-L gives the best accuracy on both datasets at about 21 FPS.

2. Models and datasets

Table 1 — Model configurations (all official COCO checkpoints; no fine-tuning)

ID	Family / paradigm	Checkpoint	Framework	Input resize
YOLO11n	YOLO; one-stage, anchor-free, DFL box head, NMS	`yolo11n.pt`	Ultralytics 8.3.200	letterbox, long side 640
YOLO11s	same family, larger width/depth	`yolo11s.pt`	Ultralytics 8.3.200	letterbox, long side 640
RT-DETR-L	DETR; hybrid CNN+transformer encoder, 300 object queries, Hungarian-trained, **no NMS**	`rtdetr-l.pt`	Ultralytics 8.3.200	**stretched** to 640×640

ID	Family / paradigm	Checkpoint	Framework	Input resize
FRCNN-MBv3-320	Faster R-CNN; two-stage, anchor-based RPN + RoI head, NMS	`FasterRCNN_MobileNet_V3_Large_320_FPN_Weights.COCO_V1`	torchvision 0.23	short side 320, long ≤ 640
FRCNN-R50-v2	Faster R-CNN; ResNet-50-FPN, improved v2 recipe	`FasterRCNN_ResNet50_FPN_V2_Weights.COCO_V1`	torchvision 0.23	short side 800, long ≤ 1333

The two YOLO sizes count as one family. The required "≥ 3 configurations from ≥ 2 families" is met three times over, by three families (YOLO, DETR, R-CNN).

COCO val2017 [6]: 5,000 images, 80 classes, official `instances_val2017.json`.

KITTI [7]: I used the 1,496-image validation split distributed with Ultralytics' KITTI dataset (images drawn from the official KITTI 2D-object *training* set, so labels exist). I did not use Ultralytics' YOLO-format labels. Instead I re-attached the **official KITTI `label_2` files**, which keep the `DontCare`, `Van`, occlusion, and truncation fields. The scored ground truth contains 5,716 cars, 921 pedestrians, and 223 trucks, plus 9,373 ignore regions.

Taxonomy mapping (`configs/kitti_taxonomy.json`). The models output COCO classes, so KITTI labels are mapped onto them:

KITTI label	Treatment
Car → car, Truck → truck, Pedestrian → person	scored
Van	ignore region for car and truck (COCO has no van; KITTI's own protocol also ignores Van when scoring Car)
Person_sitting, Cyclist	ignore region for person (a COCO model correctly boxes a cyclist's rider as *person* and the bike as *bicycle*)
Tram, Misc, DontCare	ignore region for all three classes

Ignore regions are encoded as COCO `iscrowd=1` annotations. pycocotools then neither rewards nor penalizes detections that fall inside them. All other COCO classes predicted on KITTI are discarded. For an apples-to-apples comparison I also re-scored COCO on **the same three classes only** ("COCO-3").

3. Experimental setup and hardware

Item	Value
GPU / CPU / RAM	NVIDIA GeForce RTX 3060 Ti 8 GB (driver 610.88) / AMD Ryzen 7 3700X / 32 GB
Software	Windows 11, Python 3.11.9, PyTorch 2.8.0+cu126, torchvision 0.23.0, Ultralytics 8.3.200, pycocotools 2.0.10, cuDNN 9.10
Precision / batch	FP32, batch size 1
Score threshold	0.001 (standard for mAP)
NMS IoU	0.7 (Ultralytics YOLO default), 0.5 (torchvision default), none (RT-DETR)
Detections kept	top 100 per image by score, for all models
Evaluator	pycocotools `COCOeval` bbox, IoU 0.50:0.05:0.95, maxDets 100

Item	Value
Latency	20 warm-up passes; then per image: `cuda.synchronize` → time from decoded uint8 image in host memory to boxes on CPU (pre-processing, H2D copy, network, post-processing/NMS). Disk read and JPEG/PNG decoding excluded.
Memory	`torch.cuda.max_memory_allocated` over the run (PyTorch allocator, not whole-process)

4. Commands and configuration

Everything is driven by checked-in scripts. Configurations live in `configs/models.json` and `configs/kitti_taxonomy.json`. Each run writes a JSON record (`results/runs/*.json`) containing the exact command, settings, timing boundary, and software versions. Every executed command is appended with a timestamp to `results/command_log.txt`.

```
powershell -ExecutionPolicy Bypass -File scripts/setup_windows.ps1   # venv, CUDA check, smoke test
.venv/Scripts/python.exe scripts/prepare_data.py                     # COCO val2017 + KITTI GT
conversion
.venv/Scripts/python.exe scripts/run_inference.py --model rtdetr_l --dataset kitti
.venv/Scripts/python.exe scripts/evaluate.py      --model rtdetr_l --dataset kitti
powershell -File scripts/run_all.ps1          # all 5 models x 2 datasets
powershell -File scripts/run_ablations.ps1    # Section 5.4
.venv/Scripts/python.exe scripts/failure_analysis.py --dataset kitti --draw yolo11s
.venv/Scripts/python.exe scripts/summarize.py # tables (results/summary.md) and figures
```

5. Quantitative results

Table 2 — COCO val2017, 80 classes (AP in %, latency in ms)

Model	Input	AP	AP50	AP75	AP_S	AP_M	AP_L	AR100	Latency mean / p90	FPS	Peak MiB	Published AP
YOLO11n	640	38.5	53.8	41.9	18.9	42.3	56.1	54.0	17.5 / 18.2	57.3	69	39.5
YOLO11s	640	45.5	62.0	49.2	27.6	50.1	62.5	60.2	18.4 / 19.4	54.4	110	47.0
RT-DETR-L	640	**52.2**	70.4	**56.6**	**33.4**	**56.9**	**70.5**	**67.6**	47.0 / 51.9	21.3	238	53.0
FRCNN-MBv3-320	320	23.1	38.9	23.4	2.8	22.3	44.9	32.1	29.7 / 31.5	33.7	129	22.8
FRCNN-R50-v2	800	47.0	67.9	51.4	31.0	50.8	60.4	63.0	78.5 / 82.4	12.7	672	46.7

Table 3 — KITTI val, classes person/car/truck

Model	AP	AP50	AP75	AP_S	AP_M	AP_L	AR100	Latency mean	FPS	Peak MiB
YOLO11n	26.8	52.4	24.4	12.4	24.6	41.5	41.3	17.2	58.1	52
YOLO11s	31.0	57.9	30.6	18.2	29.0	45.0	44.4	16.6	60.3	83
RT-DETR-L	**38.3**	66.9	**38.8**	**30.3**	**34.8**	**52.8**	52.9	44.7	22.4	238
FRCNN-MBv3-320	16.1	34.5	14.1	0.2	9.3	38.6	26.8	29.6	33.8	121
FRCNN-R50-v2	36.5	**67.2**	37.4	25.5	33.7	48.6	**55.8**	62.6	16.0	442

Per-class AP and the full metric set for every run are in `results/metrics/*.json`. All tables are regenerated by `scripts/summarize.py` into `results/summary.md`.

5.1 Analysis 1 — Cross-dataset generalization

Table 4 — COCO-3 → KITTI (AP@[.50:.95] per class)

Model	COCO-3 AP	KITTI AP	Δ	person	car	truck
YOLO11n	39.4	26.8	−12.6	52.7 → 25.2	37.1 → 42.8	28.5 → 12.4
YOLO11s	46.3	31.0	−15.3	58.6 → 27.7	44.8 → 47.1	35.4 → 18.3
RT-DETR-L	52.3	38.3	−14.0	61.5 → 29.7	49.8 → 51.6	45.6 → 33.7
FRCNN-MBv3-320	20.6	16.1	−4.4	29.9 → 15.5	13.7 → 25.0	18.0 → 7.8
FRCNN-R50-v2	50.8	36.5	−14.3	59.7 → 30.0	50.0 → 49.3	42.7 → 30.2

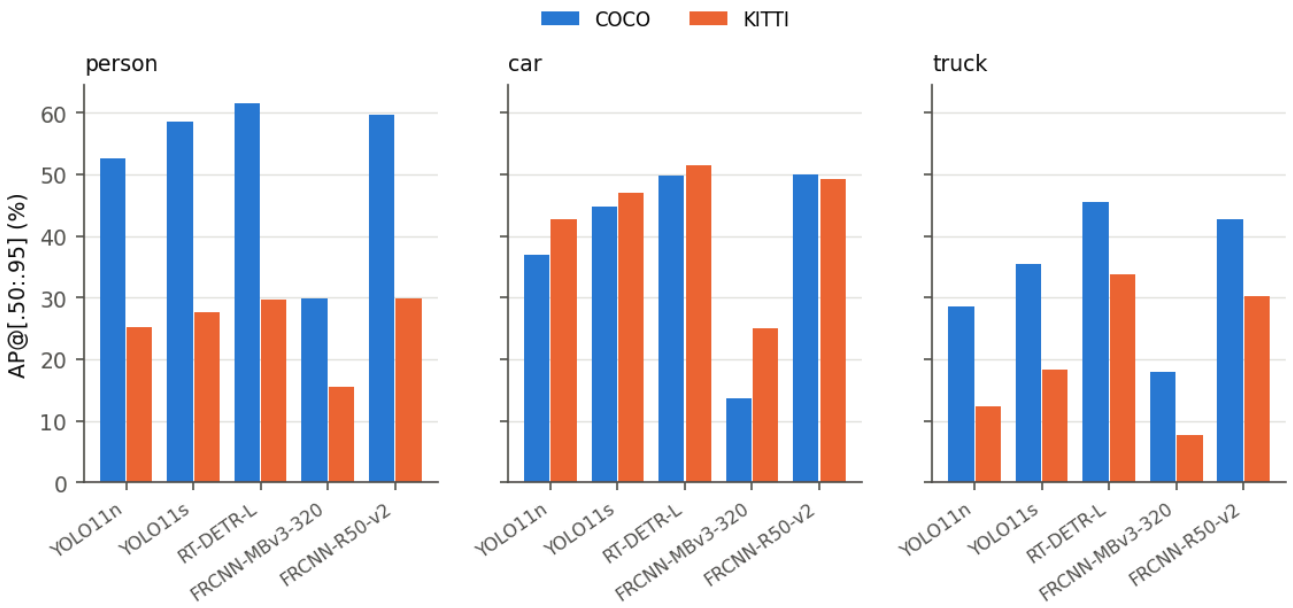


Figure 1. Per-class AP on COCO (restricted to the shared classes) and on KITTI. Cars transfer; persons and trucks do not.

Which classes degrade most? The aggregate drop (−13 to −15 AP for the three stronger models) hides three different behaviors:

- **Car transfers well (Δ ≈ −1 to +6 AP).** KITTI cars are mostly unoccluded, mid-sized vehicles seen from a dash camera, an easy subset of COCO's car distribution. In COCO, 62% of car instances are *small* (< 32^2 px); in KITTI only 16% are.
- **Person loses about half its AP (−27 to −32 AP), and this is not a scale effect.** KITTI pedestrians are mostly mid-sized (median box height 87 px; only 19% small, versus 40% in COCO). AP50 falls by about 19 points as well (YOLO11s 81.4 → 62.7, RT-DETR 85.8 → 66.1), so both recall and localization suffer. The likely causes are partial occlusion behind parked cars and sidewalk clutter. The failure counts support this: miss rate rises from 10–13% for fully visible objects to 54–64% for largely occluded ones, for the three stronger models. Annotation convention also contributes: KITTI pedestrian boxes are tight and exclude carried objects. That convention costs most at strict IoU thresholds, consistent with AP@[.50:.95] for person falling about 30 points while AP50 falls only about 19.
- **Truck collapses (−10 to −16 AP), largely because of taxonomy.** For every KITTI truck I recorded the class of the highest-IoU prediction (score ≥ 0.3, IoU ≥ 0.5). Many were predicted as COCO **train** or **bus**: 75 of 223 for YOLO11s, 53 for RT-DETR, and 100 for FRCNN-R50-v2. These detections are spatially correct but labeled with a class that has no KITTI counterpart. COCO's *truck* is dominated by pickups and box trucks seen from the side. KITTI's *Truck* is mostly semi-trailers and delivery trucks seen from behind, which resemble the front or back of a bus or train.

Which scales degrade most? Comparing COCO-3 with KITTI on the same three classes: for YOLO11s, AP_S falls from 27.7 to 18.2 and AP_M from 55.6 to 29.0. For RT-DETR, AP_S changes from 33.8 to 30.3 and AP_M from 61.0 to 34.8. Medium objects lose the most in absolute terms, because the occluded pedestrians and trucks above are mostly medium-sized.

The KITTI-specific breakdowns (Figure 2) show monotonic degradation with occlusion for every model. Under KITTI's "moderate" filter (height ≥ 25 px, occlusion ≤ 1, truncation ≤ 0.3), RT-DETR-L scores 42.8, FRCNN-R50-v2 40.9, YOLO11s 35.6, YOLO11n 31.7, and FRCNN-MBv3-320 18.6 AP.

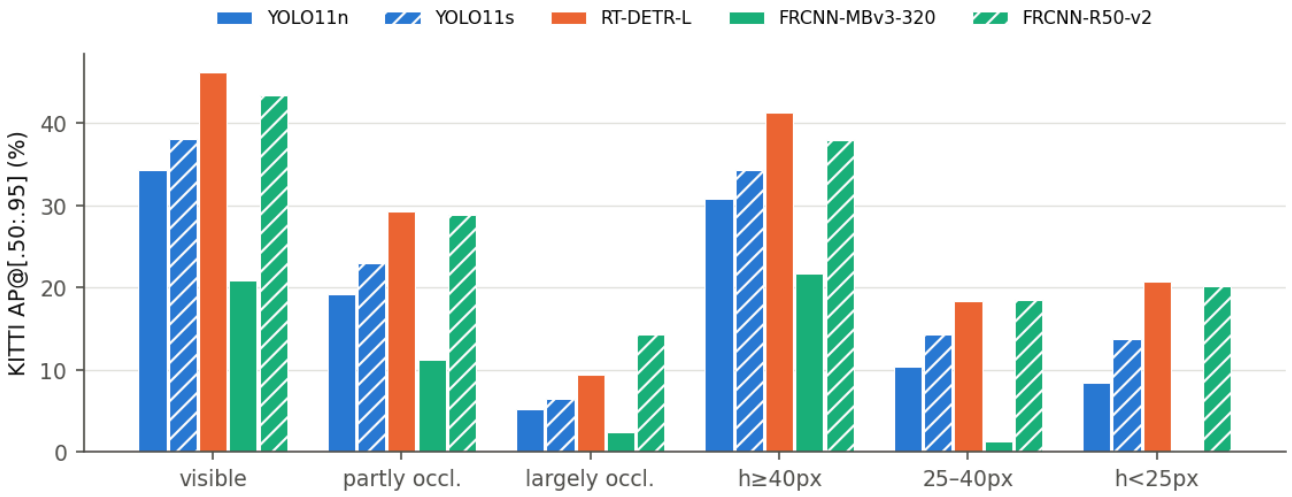


Figure 2. KITTI AP for subsets by occlusion level (KITTI labels 0/1/2) and by box height. Objects outside each subset become ignore regions.

5.2 Analysis 2 — Speed–accuracy trade-off

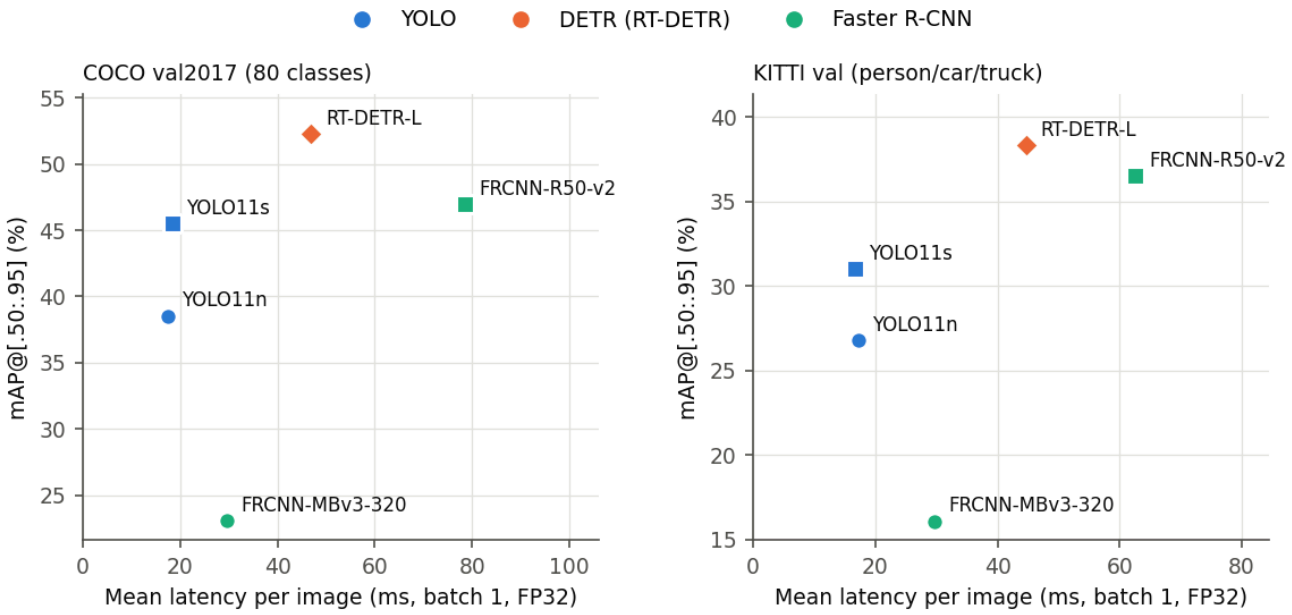


Figure 3. Accuracy vs mean per-image latency on the RTX 3060 Ti (FP32, batch 1, end-to-end excluding decode). Color = family; marker = model size.

- **Batch-1 latency is overhead-bound for small models.** YOLO11s has about 3× the FLOPs of YOLO11n (21.5 vs 6.5 GFLOPs at 640 px, per Ultralytics) but costs only 0.9 ms more on COCO (18.4 vs 17.5 ms) and is *faster* on KITTI. Python pre/post-processing and kernel-launch overhead dominate. **YOLO11s therefore strictly dominates YOLO11n here** (+7 AP for ~1 ms).

- **The two-stage design costs latency.** FRCNN-MBv3-320 runs at a tiny 320 px input yet takes 29.7 ms, more than either YOLO, because the RPN, RoI Align over the proposals (150 for this variant; 1,000 for R50), and per-class NMS add sequential stages. FRCNN-R50-v2 needs 78.5 ms and 672 MiB for 47.0 AP: it is 1.7× slower than RT-DETR-L, uses 2.8× more memory, and is 5 AP less accurate.
- **RT-DETR-L is on the Pareto front** (52.2 AP, 47 ms). It costs about 2.6× YOLO11s's latency for +6.7 AP on COCO and +7.3 AP on KITTI.
- **Latency depends on the dataset's image shape.** FRCNN-R50-v2 is faster on KITTI (62.6 ms) than on COCO (78.5 ms) because the 1333-px long-side cap shrinks KITTI frames to 1333×402 (0.54 MP) versus about 800×1066 (0.85 MP) for COCO. YOLO letterboxes KITTI to 640×224 (minimal stride-32 padding), whereas RT-DETR always processes 640×640.

5.3 Analysis 3 — Architecture comparison: YOLO vs RT-DETR vs Faster R-CNN

- **Post-processing and duplicates.** YOLO11 and Faster R-CNN predict dense, redundant candidates and rely on NMS. RT-DETR is trained with one-to-one Hungarian matching [3,4], so each object should be claimed by one query and NMS is unnecessary. The failure counts agree: at each model's best-F1 threshold on KITTI, RT-DETR produced **30 duplicate detections versus 86 for YOLO11s and 109 for YOLO11n**. FRCNN-R50-v2 (15) also has few duplicates, but only because its best threshold is a very high 0.9.
- **Score calibration differs by family.** The F1-optimal thresholds were 0.35 (YOLO), 0.55 (RT-DETR), and 0.5–0.9 (Faster R-CNN). One shared confidence threshold, as is common in deployment, would therefore favor some families over others. This is why I compare failures at each model's own best-F1 operating point.
- **Localization quality.** On KITTI, AP75/AP50 is 0.58 for RT-DETR, 0.56 for FRCNN-R50-v2, 0.53 for YOLO11s, and 0.41 for FRCNN-MBv3-320. *Localization* ($0.1 \le \text{IoU} < 0.5$ with a same-class object) is the largest false-positive type for every model. Its main source is the rows of overlapping parked cars typical of KITTI streets (Figure 4).
- **Recall on hard objects.** FRCNN-R50-v2 has the highest AR100 on KITTI (55.8) and the best AP on largely occluded objects (14.3 vs 9.5 for RT-DETR). Its 1,000 RPN proposals and higher input resolution (1333×402 vs 640×193 for YOLO) retain more heavily occluded candidates.

5.4 Ablations: input geometry and detection budget

Table 5 — Ablations (KITTI rows; one COCO row for the detection budget)

Configuration	Dataset	AP	AP50	AP_S	AP_M	AP_L	Latency ms	Peak MiB
RT-DETR-L, stretched to 640×640 (default)	KITTI	**38.3**	66.9	30.3	34.8	**52.8**	44.7	238
RT-DETR-L, pad to square → 640×193 content	KITTI	37.2	66.8	**33.5**	34.8	48.9	49.0	238
YOLO11s, 640 (default)	KITTI	31.0	57.9	18.2	29.0	45.0	16.6	83
YOLO11s, 1280	KITTI	**35.6**	64.0	**32.8**	33.0	45.6	18.5	123
YOLO11n, 640 (default)	KITTI	26.8	52.4	12.4	24.6	41.5	17.2	52
YOLO11n, 1280	KITTI	**33.1**	61.2	**21.9**	32.0	43.8	19.5	78
YOLO11s, top-100 detections (default)	COCO	45.5	62.0	27.6	50.1	62.5	18.4	110
YOLO11s, top-300 detections	COCO	45.7	62.3	27.8	50.2	62.5	17.9	110

Input resolution on wide frames (YOLO). At 1280 px, a KITTI frame becomes 1280×387 instead of 640×193. Small-object AP rises from 18.2 to 32.8 for YOLO11s and from 12.4 to 21.9 for YOLO11n, while large-object AP barely moves. Latency rises only about 2 ms because the image is still thin, so at batch 1 the GPU stays

overhead-bound. YOLO11n at 1280 (33.1 AP, 19.5 ms) beats YOLO11s at 640 (31.0 AP, 16.6 ms). On wide driving images, **input resolution is a cheaper lever than model size**.

Stretching vs aspect preservation (RT-DETR). Ultralytics' RT-DETR predictor resizes every image to 640×640 without preserving aspect ratio (`LetterBox(auto=False, scale_fill=True)` in `ultralytics/models/rtdetr/predict.py`). A 1242×375 KITTI frame is therefore squeezed horizontally by 0.52 and stretched vertically by 1.71. I expected this distortion, or the extra vertical pixels it creates, to explain RT-DETR's KITTI lead. The ablation contradicts that. Padding to a square first keeps proportions but leaves only 640×193 pixels of content, and it *raises* small-object AP (30.3 → 33.5) while lowering large-object AP (52.8 → 48.9), for −1.1 AP overall. RT-DETR's small-object advantage over YOLO11s at the same 640×193 content (33.5 vs 18.2) therefore comes from the model, not the resize. The stretch mainly helps large, nearby objects, plausibly because they then fill the 640×640 canvas in a way closer to COCO's object scale.

Detection budget (COCO). Keeping the top 300 detections per image instead of 100 changes YOLO11s from 45.51 to 45.65 AP. That covers only 0.1 of the 1.5-point gap to the published 47.0. A remaining protocol difference I verified in the source: Ultralytics' validator runs NMS with `multi_label=True` (`ultralytics/models/yolo/detect/val.py`), so one box can contribute a detection for several classes. The `predict()` path I used keeps one label per box. I did not run the validator, so the size of this effect is not measured.

6. Failure analysis

For each model I picked the score threshold that maximizes F1 at IoU 0.5 and matched detections greedily to ground truth. I classified each unmatched detection as a *duplicate*, *class confusion*, *localization* error, or *background* false positive, and split misses by box height and occlusion (`results/failure_analysis_kitti.json`).

Model (KITTI)	Thr	F1	Background FP	Localization FP	Duplicate	Class conf.	Miss rate h<25 / 25–40 / 40–100 / ≥100 px
YOLO11n	0.35	0.72	207	457	109	35	0.64 / 0.49 / 0.35 / 0.14
YOLO11s	0.35	0.78	226	459	86	46	0.45 / 0.35 / 0.28 / 0.14
RT-DETR-L	0.55	**0.82**	161	419	30	38	0.29 / 0.27 / 0.26 / 0.14
FRCNN-MBv3-320	0.50	0.54	219	698	75	48	0.96 / 0.84 / 0.50 / 0.21
FRCNN-R50-v2	0.90	0.81	310	593	15	5	0.24 / 0.26 / 0.25 / 0.15

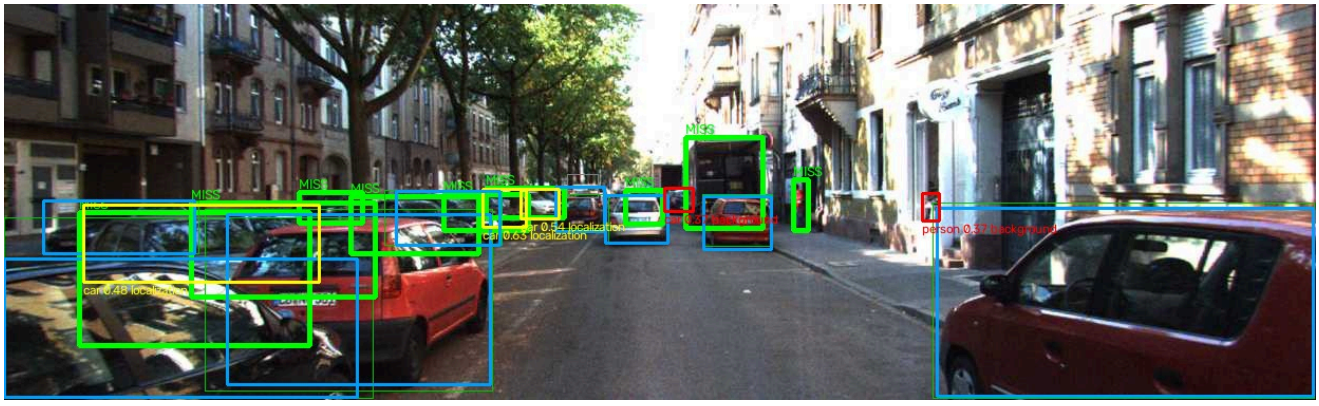


Figure 4. YOLO11s on KITTI image 000146 (threshold 0.35). Blue = true positive; thick bright green with "MISS" = missed ground truth; yellow = localization error; red = background FP; gray = ignore region. Failure modes: (1) a row of parked cars occluding one another produces merged or shifted boxes (yellow) and misses; (2) distant cars under 25 px tall in the vanishing-point region are missed; (3) the truck in the center is detected as *truck* but only at score 0.23, below the operating threshold; (4) a narrow pedestrian beside the parked cars (right of center) is missed, and a small object by a doorway is detected as a person (red).

Failure modes, with evidence.

1. **Small, distant objects.** Miss rate for objects under 25 px tall is 45–96% for YOLO and FRCNN-MBv3, which see KITTI at 640×193, versus 24–29% for FRCNN-R50-v2 and RT-DETR. Running YOLO at 1280 px closes most of this gap (Section 5.4).
2. **Occlusion.** Miss rate grows from 10–20% (fully visible) to 54–72% (largely occluded) for every model except FRCNN-MBv3-320, which is worse throughout.
3. **Crowded parked cars.** These cause localization errors, the largest FP category for all five models.
4. **Taxonomy mismatch for trucks.** Detections are spatially correct but labeled bus or train, as quantified in Section 5.1.
5. **Background false positives.** These are 21–34% of false positives at the operating point (for example, the object by the doorway predicted as a person in Figure 4). I did not categorize them further, so their specific causes are not quantified here.
6. **Input resolution.** FRCNN-MBv3-320 at 320 px misses 96% of objects under 25 px tall, so small-object AP is near zero on both datasets (2.8 COCO, 0.2 KITTI).

7. Conclusions

- My measured COCO accuracy matches published results (≤ 1.5 AP difference; for YOLO the remaining gap is a validator protocol difference, not the detection budget). The pipeline can therefore be trusted for the cross-domain comparison.
- Zero-shot transfer from COCO to KITTI costs about 13–15 AP. Nearly all of the loss comes from **person** (occlusion, annotation convention) and **truck** (taxonomy: KITTI trucks predicted as bus or train). Cars transfer with no loss. A per-class view is essential; the aggregate number would suggest uniform degradation that does not exist.
- On wide driving frames, **input resolution** is a first-order factor for YOLO: doubling it adds 4.6–6.3 AP for about 2 ms, more than moving from the n to the s model. A nominal "640" input hides very different preprocessing across frameworks (letterbox vs stretch). The ablation shows that RT-DETR's stretch helps large objects but is not the source of its small-object lead.
- For this GPU at batch 1, **RT-DETR-L** is the most accurate configuration (38.3 KITTI AP at about 22 FPS). **YOLO11s at 1280 px** (35.6 AP at about 54 FPS) is the best choice above 50 FPS. At equal input size YOLO11s dominates YOLO11n. Faster R-CNN R50-v2 has the best recall on occluded objects but is the slowest and most memory-hungry.
- **Limitations and next steps.** I did no fine-tuning, so KITTI fine-tuning (or a class-agnostic vehicle mapping that counts bus/train/truck as one class) would separate taxonomy loss from true domain shift. I used FP32 only; TensorRT/FP16 could change the latency ranking and was not tested. The 1,496-image KITTI split has only 223 trucks, so truck AP has high variance.

8. References

1. S. Ren, K. He, R. Girshick, J. Sun. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.* NeurIPS 2015.
2. T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie. *Feature Pyramid Networks for Object Detection.* CVPR 2017.
3. N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko. *End-to-End Object Detection with Transformers (DETR).* ECCV 2020.
4. Y. Zhao, W. Lv, S. Xu, et al. *DETRs Beat YOLOs on Real-time Object Detection (RT-DETR).* CVPR 2024.

5. J. Redmon, S. Divvala, R. Girshick, A. Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. CVPR 2016.
6. T.-Y. Lin, M. Maire, S. Belongie, et al. *Microsoft COCO: Common Objects in Context*. ECCV 2014.
7. A. Geiger, P. Lenz, R. Urtasun. *Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite*. CVPR 2012.
8. A. Howard, M. Sandler, G. Chu, et al. *Searching for MobileNetV3*. ICCV 2019.
9. Y. Li, S. Xie, X. Chen, P. Dollár, K. He, R. Girshick. *Benchmarking Detection Transfer Learning with Vision Transformers* (basis of torchvision's Faster R-CNN v2 recipe). arXiv:2111.11429, 2021.
10. C. Feng, Y. Zhong, Y. Gao, M. Scott, W. Huang. *TOOD: Task-aligned One-stage Object Detection* (assigner used by YOLOv8/YOLO11). ICCV 2021.
11. X. Li, W. Wang, L. Wu, et al. *Generalized Focal Loss* (Distribution Focal Loss used in YOLO11's box head). NeurIPS 2020.
12. Ultralytics. *YOLO11 and RT-DETR documentation* and source (`ultralytics/models/rtdetr/predict.py`), v8.3.200. https://docs.ultralytics.com, https://github.com/ultralytics/ultralytics
13. PyTorch. *torchvision.models.detection documentation* (Faster R-CNN weights and COCO metrics). https://pytorch.org/vision/stable/models.html
14. COCO API (pycocotools) — `cocoeval.py` evaluation protocol. https://github.com/cocodataset/cocoapi
15. K. Liu. *DeepDataMiningLearning — ngdet tutorial* (course starting point). https://github.com/lkk688/DeepDataMiningLearning